TECHNICAL REPORT · MACHINE LEARNING

Fine-tuning small open models for text-to-SQL on a company database

LoRA fine-tuning of Qwen3.5-4B and Llama 3.2 3B on the Olist e-commerce database, evaluated on a locked 150-question test

Document ID	OLIST-T2S-TR-001
Version	1.0 (final)
Date	17 October 2026
Author	Ajay Karthick
Data classification	Public. All data comes from Olist's public dataset; no customer personal data is used.
Repository	`olist-text2sql` (commit `35e57e3`); test set fingerprint `53cd9d63…8693d8`

1 Executive summary

This project tested whether a small open-source language model, fine-tuned on a few hundred examples, can reliably turn business questions into correct SQL for one company's database. Two models were fine-tuned with LoRA and scored on a 150-question test that was locked before any model was run.

52 → 78
Qwen3.5-4B, correct of 150, untrained → fine-tuned (mean of 3 runs)

18 → 51.7
Llama 3.2 3B, correct of 150, untrained → fine-tuned (mean of 3 runs)

6 of 6
training runs beat their untrained model (McNemar $p \le 0.0002$)

~2 h
of single-GPU training per Qwen run (NVIDIA L4)

Findings

- Fine-tuning works and is repeatable. Every one of the 6 training runs beat its untrained model, and the gain held across three random seeds.
- Qwen3.5-4B is the stronger base model. It beat Llama 3.2 3B before training (52 vs 18) and after it, winning all three same-seed comparisons.
- The fine-tuned model is not ready to run unsupervised. It answers 52% of all questions and 25% of hard ones, against 95% for a large hosted model (Gemini 3.8 Flash). It is suited to drafting SQL that an analyst reviews.
- Training-data design mattered more than training time. The first attempt memorised its data and gave no clear gain; a more varied dataset with gentler settings gave a clear one.

2 Background

Most business questions about an online marketplace (which sellers are late, where freight is expensive, which categories get poor reviews) can be answered from its order database, but only by someone who writes SQL and knows where the data's traps are. A model that writes correct SQL for that specific database would let more people get answers without waiting for an analyst.

Large hosted models already write SQL well, but they send company data to an outside service and are billed per request. A small open model can run on the company's own hardware. The question is whether a small model can be made good enough on one company's schema with a modest amount of training.

3 Objectives and non-goals

Objectives	Non-goals
• Measure how much LoRA fine-tuning improves a small open model's SQL on one real schema. • Compare two open base models under identical conditions. • Place the results against a large hosted model as a reference point. • Report results that hold up statistically, including run-to-run variation.	• Beat large hosted models. • Build a production service, user interface or chat assistant. • Generalise across many databases (one schema only). • Tune for speed or serving cost.

4 System overview

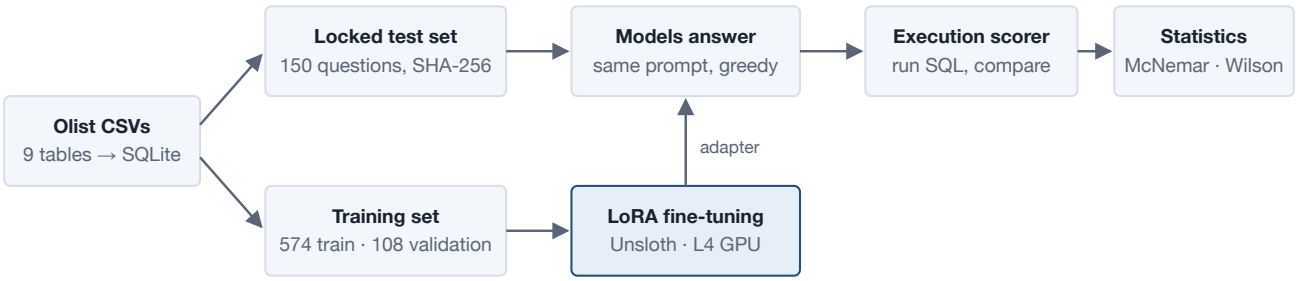


Figure 1. Pipeline. The test set is fixed before training; the training set is checked so that no example resembles a test question. Every model, trained or not, is scored by the same code.

5 Data

- **Source database.** Olist's public Brazilian e-commerce dataset: 99,441 orders placed between September 2016 and October 2018, in 9 tables (orders, order items, payments, reviews, customers, sellers, products, category names, geolocation). It is loaded into SQLite with 18 indexes. Table names match the Spider 2.0 benchmark's copy of the same data. [1]
- **Test set (150 questions).** 139 questions were written for this project across the 9 business themes listed in Olist's own README, and reviewed in batches. The other 11 are the Olist questions published in the Spider

2.0 benchmark [2], used for testing only.

- **Data traps.** Questions deliberately test known pitfalls in the real data, for example that an order has one row per item, that delivery dates are missing for undelivered orders, and that some orders have several reviews.
- **Training set.** 574 training and 108 validation examples, generated from 118 question "families" written with Claude (Anthropic). Every query was run against the database and cross-checked by a second, independently written query. Whole families are held out for validation.
- **Leakage controls.** Each training question is compared with all 150 test questions: text similarity below 0.75, word overlap below 0.6, and no identical results. No output from the reference model (Gemini) was used for training.

6 Evaluation design

Metric

Execution accuracy. An answer is correct only if the generated SQL runs and returns the same result as the reference query. Results are compared on values, not column names; row order counts only when the question asks for an order; numbers are rounded to 2 decimal places. The 11 Spider questions use Spider's own grading rule. Generated SQL runs against a read-only connection, must be a single `SELECT`, and is stopped after 30 seconds.

Controls

- The test files were fingerprinted with SHA-256 before any model was run and verified before every scoring run.
- All models receive the same prompt (instructions plus the database schema) and decode greedily, so answers are deterministic.
- The untrained baseline for each model was run on the same GPU and code as its fine-tuned version.
- Checkpoints are chosen on the validation set only. The test set is never used to choose anything.
- Each model was trained three times (seeds 3407, 42 and 1234), and every run is reported.

Statistical method and decision rules

Two runs on the same 150 questions are compared with McNemar's exact test [4], which uses only the questions where the runs disagree. Single scores carry 95% Wilson intervals [5]. At 150 questions, a paired difference of about ±9 points can arise by chance. Three rules were written down before the repeat runs:

1. All three seeds are reported for each model, never only the best.
2. Fine-tuning counts as working for a model only if every seed beats its untrained model with $p < 0.05$.
3. Qwen counts as better than Llama only if its mean is higher and it wins every same-seed pair.

7 Training

Both models were fine-tuned with LoRA [3], which freezes the original weights and trains small low-rank matrices added to the attention and feed-forward layers. Training used Unsloth on one NVIDIA L4 GPU in bf16 (Google Colab). Loss is computed on the SQL answer only; the prompt is identical to the one used at test time.

Setting	Round 1 (Qwen only)	Round 2 (both models, all seeds)
Training data	540 examples, 63 families	574 examples, 118 families, several SQL styles per family
Learning rate, epochs	2e-4, 3 epochs	1e-4, 2 epochs
Weight decay, LoRA dropout	0.001, 0	0.01, 0.05
Validation	76 examples, after each epoch	108 examples, every half epoch
Unchanged	LoRA rank 16, alpha 16; batch 2 × 4 accumulation; 10 warm-up steps; adamw_8bit; linear decay	

Round 1 memorised its training data: training loss fell to 0.0003 and the model began copying the training SQL's alias style, which broke 16 questions it had answered correctly before. Round 2 changed the data and settings following Unsloth's guidance on overfitting [6]; final training loss stayed between about 0.06 and 0.12.

8 Results

Run	Correct /150	Accuracy (95% CI)	Gain, points (95% CI)	Fixed / broken	p
Qwen3.5-4B, untrained	52	34.7% (27.5–42.6)	–	–	–
Qwen3.5-4B, round 1	63	42.0% (34.4–50.0)	+7.3 (–1.2 to +15.8)	27 / 16	0.13
Qwen3.5-4B, seed 3407	80	53.3% (45.4–61.1)	+18.7 (+11.0 to +26.4)	34 / 6	< 0.0001
Qwen3.5-4B, seed 42	75	50.0% (42.1–57.9)	+15.3 (+7.8 to +22.9)	30 / 7	0.0002
Qwen3.5-4B, seed 1234	79	52.7% (44.7–60.5)	+18.0 (+10.4 to +25.6)	33 / 6	< 0.0001
Llama 3.2 3B, untrained	18	12.0% (7.7–18.2)	–	–	–
Llama 3.2 3B, seed 3407	50	33.3% (26.3–41.2)	+21.3 (+14.0 to +28.6)	35 / 3	< 0.0001
Llama 3.2 3B, seed 42	57	38.0% (30.6–46.0)	+26.0 (+18.3 to +33.7)	42 / 3	< 0.0001
Llama 3.2 3B, seed 1234	48	32.0% (25.1–39.8)	+20.0 (+12.8 to +27.2)	33 / 3	< 0.0001
Gemini 3.8 Flash (reference)	142	94.7% (89.8–97.3)	–	–	–

Table 1. All runs. Gain, fixed/broken and p are against the same model untrained (McNemar exact test). Gemini was not trained and is shown for reference only.

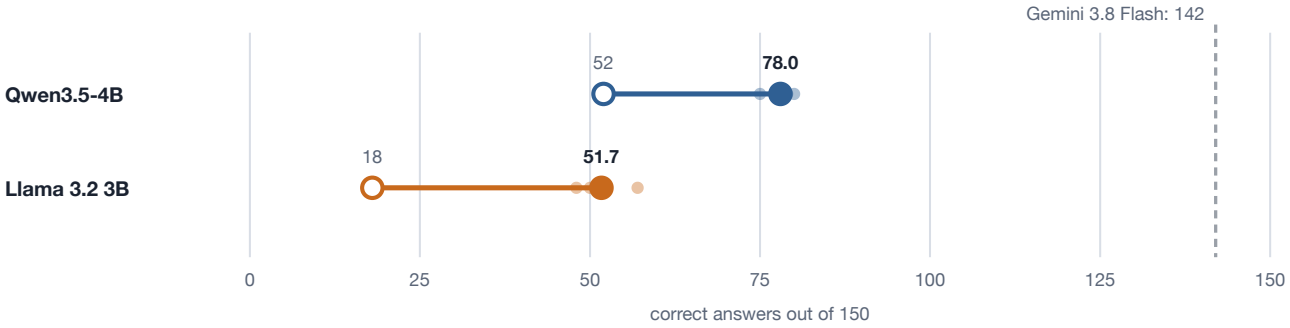


Figure 2. Untrained (hollow) against fine-tuned mean (solid); faint dots are the three individual runs. The dashed line is the large reference model.

Qwen against Llama, same seed

Seed	Llama	Qwen	Qwen lead, points (95% CI)	Right only by Qwen / Llama	p
3407	50	80	+20.0 (+12.2 to +27.8)	36 / 6	< 0.0001
42	57	75	+12.0 (+4.9 to +19.1)	25 / 7	0.0021
1234	48	79	+20.7 (+13.0 to +28.4)	36 / 5	< 0.0001

Table 2. Head-to-head on the same 150 questions. All three decision rules from section 6 pass: every seed of both models beats its untrained model, and Qwen wins every pair. Seed variation is 5 questions for Qwen (75–80) and 9 for Llama (48–57).

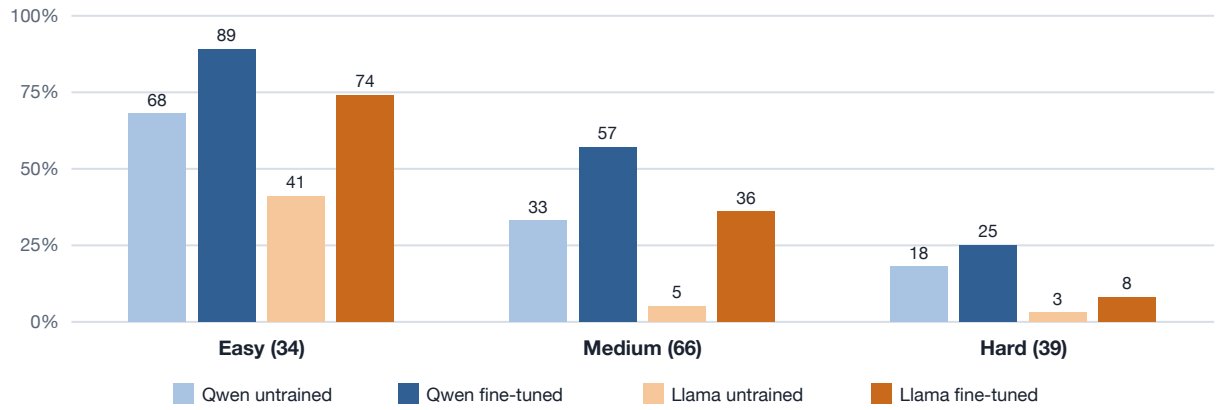


Figure 3. Accuracy (%) by difficulty on the 139 project questions; question counts in brackets. Fine-tuned values pool all three runs.

By data trap

Trap (questions)	Qwen untrained	Qwen fine-tuned	Llama untrained	Llama fine-tuned
Missing delivery date (16)	19%	62%	0%	33%
Which date to use (44)	39%	53%	2%	33%
Order status filter (69)	33%	48%	9%	30%
One row per item (32)	31%	44%	9%	21%
Customer vs person ID (8)	0%	25%	0%	21%
Several reviews per order (12)	17%	33%	8%	14%

Table 3. Accuracy on questions that test each trap (traps with at least 8 questions; a question can test several traps).

9 Error analysis

Outcome (answers per run, of 150)	Qwen untrained	Qwen fine-tuned	Llama untrained	Llama fine-tuned
Correct	52	78.0	18	51.7
Ran, wrong result	55	43.0	54	65.7
Wrong column or table	27	19.0	34	19.3
Syntax error	9	4.7	34	8.0
Other SQL error	3	2.3	5	4.7
Function from another SQL dialect	3	1.0	5	0.7
Exceeded 30 s	1	2.0	0	0

Table 4. Every answer falls into exactly one outcome. Fine-tuned values are means of three runs.

- **Training mostly removes SQL that does not run.** Llama's syntax errors fell from 34 to 8 per run and invented columns from 34 to 19; Qwen's invented columns fell from 27 to 19.
- **Logic errors remain.** For Llama, many queries that no longer crash now run and return the wrong result (54 → 65.7). The main remaining weakness for both models is query logic on hard questions. The traps that stay hardest after training are one row per item, which date to use, and the order status filter (Table 3).
- **Invented columns are still the largest error group** after wrong results, at about 19 per run for both models.

10 Cost and performance

Model	Training time per run	Adapter size	Median time per answer
Qwen3.5-4B (4B parameters)	107–115 min	≈ 120 MB	10.8 s
Llama 3.2 3B (3B parameters)	49–56 min	≈ 90 MB	4.4 s
Gemini 3.8 Flash (hosted)	not trained	–	5.5 s (API, incl. network)

Table 5. One NVIDIA L4 GPU (Google Colab). Answer times are for one question at a time with standard Hugging Face generation and no batching or serving optimisation, so they overstate production latency. Training time includes the validation passes.

11 Risks and limitations

- **Test difficulty.** The project questions state exactly what is wanted, so the large reference model answers all 139 of them correctly. The test shows a small model closing the gap with a large one, not beating it.
- **Hard questions.** Accuracy on hard questions is 25% (Qwen). The 11 Spider questions, which are less precisely worded, stayed at 0 or 1 correct for every small-model run.
- **Scope.** One database and one SQL dialect. Results do not show how the method transfers to another schema.
- **Training data origin.** Training examples were written with an AI assistant and validated by execution, not collected from real users, so real questions may be phrased differently.
- **Operational safety.** Wrong SQL that runs returns a plausible-looking wrong number. Any use needs read-only access (as in this evaluation), query time limits, and human review of results.
- **Test size.** With 150 questions, differences under about 9 points cannot be separated from chance. The main results are well above that level.

12 Conclusions and recommendations

1. LoRA fine-tuning on about 600 checked, schema-specific examples reliably improves a small open model's SQL on that schema: +26 questions for Qwen and +34 for Llama on average, consistent across seeds.
2. Qwen3.5-4B is the recommended base model among those tested.
3. The fine-tuned model should be used to draft SQL for an analyst to review, not to answer business users directly.
4. The next improvement should target query logic on hard questions, through more hard training families and a harder, less precisely worded evaluation set, rather than longer training.

13 Next steps

Item	Owner	Status
Publish the Qwen adapter with a model card (evaluation results and limitations) on Hugging Face	Ajay Karthick	PLANNED
Publish code, locked test set and results in a public GitHub repository	Ajay Karthick	PLANNED
Publish the results dashboard on Tableau Public	Ajay Karthick	PLANNED
Build a second, harder evaluation set (vague, real-world phrasing), locked before use	Ajay Karthick	PROPOSED
Results dashboard (6 views) and this report	Ajay Karthick	DONE

Appendix A · Reproducibility

Step	Command or file
Build the database	`python scripts/build_db.py`
Verify the locked test set	`python scripts/lock_test_set.py verify` (50 files, SHA-256)
Export the shared prompts	`python scripts/export_prompts.py` → `results/prompts.jsonl`
Build the training set	`python scripts/make_train_set.py` → `data/train_r2/`
Build a training notebook	`python scripts/make_finetune_notebook.py --round 2|llama [--seed N]`
Score a run / compare two runs	`python scripts/score.py <run>`, `--compare <a> <b>`
Rebuild the dashboard data	`python scripts/export_tableau_data.py`, then `make_tableau_workbook.py`

Every run's predictions, scores and training log are stored under `results/<run>/`. Notebooks embed their data and check it by SHA-256. Adapters are stored outside version control because of their size.

Appendix B · Licences and terms

Component	Licence or terms
Olist dataset	MIT licence [1]
Spider 2.0 questions and grading code	MIT licence [2]
Qwen3.5-4B	Apache-2.0 [7]
Llama 3.2 3B Instruct	Llama 3.2 Community License: a distributed derived model's name must begin with "Llama", and "Built with Llama" must be displayed [8]
Training examples	Written with Claude (Anthropic) and validated by running every query against the database.
Gemini 3.8 Flash	Used for evaluation only. Its outputs were never used for training, in line with Google's terms.

Appendix C · Glossary

LoRA	Low-Rank Adaptation. Trains small added matrices instead of the whole model, so training fits on one GPU and the result is a small adapter file.
Execution accuracy	The share of questions where the generated SQL returns the same result as the reference query.
Seed	The number that fixes the random choices in training (starting values, data order, dropout). Different seeds give different runs of the same recipe.
Validation set	Held-back training-style examples used to choose the checkpoint, so the test set is never used for any choice.
McNemar's test	A paired test for two systems scored on the same items; it uses only the items where they disagree.
Data trap	A property of the real data that makes a question easy to answer wrongly, such as missing dates or one row per item.

References

1. Olist. Brazilian E-Commerce Public Dataset. github.com/olist/work-at-olist-data
2. Lei, F. et al. (2024). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. github.com/xlang-ai/Spider2
3. Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685
4. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2).
5. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158).
6. Unsloth. LoRA hyperparameters guide. unsloth.ai/docs
7. Qwen. Qwen3.5-4B model card. huggingface.co/Qwen/Qwen3.5-4B
8. Meta. Llama 3.2 Community License. github.com/meta-llama/llama-models